

Argue that the expected running time of RANDOMIZED-SELECT does not depend on the order of the elements in its input array $A[p : r]$. That is, the expected running time is the same for any permutation of the input array $A[p : r]$. (*Hint*: Argue by induction on the length n of the input array.)

9.3 Selection in worst-case linear time

We'll now examine a remarkable and theoretically interesting selection algorithm whose running time is $\Theta(n)$ in the worst case. Although the RANDOMIZED-SELECT algorithm from Section 9.2 achieves linear expected time, we saw that its running time in the worst case was quadratic. The selection algorithm presented in this section achieves linear time in the worst case, but it is not nearly as practical as RANDOMIZED-SELECT. It is mostly of theoretical interest.

Like the expected linear-time RANDOMIZED-SELECT, the worst-case linear-time algorithm SELECT finds the desired element by recursively partitioning the input array. Unlike RANDOMIZED-SELECT, however, SELECT *guarantees* a good split by choosing a provably good pivot when partitioning the array. The cleverness in the algorithm is that it finds the pivot recursively. Thus, there are two invocations of SELECT: one to find a good pivot, and a second to recursively find the desired order statistic.

The partitioning algorithm used by SELECT is like the deterministic partitioning algorithm PARTITION from quicksort (see Section 7.1), but modified to take the element to partition around as an additional input parameter. Like PARTITION, the PARTITION-AROUND algorithm returns the index of the pivot. Since it's so similar to PARTITION, the pseudocode for PARTITION-AROUND is omitted.

The SELECT procedure takes as input a subarray $A[p : r]$ of $n = r - p + 1$ elements and an integer i in the range $1 \leq i \leq n$. It returns the i th smallest element of A . The pseudocode is actually more understandable than it might appear at first.

```
SELECT( $A, p, r, i$ )
```

```

1 while  $(r - p + 1) \bmod 5 \neq 0$ 
2   for  $j = p + 1$  to  $r$                                 // put the minimum into  $A[p]$ 
3     if  $A[p] > A[j]$ 
4       exchange  $A[p]$  with  $A[j]$ 
5   // If we want the minimum of  $A[p : r]$ , we're done.
6   if  $i == 1$ 
7     return  $A[p]$ 
8   // Otherwise, we want the  $(i - 1)$ st element of  $A[p + 1 : r]$ .
9    $p = p + 1$ 
10   $i = i - 1$ 
11  $g = (r - p + 1) / 5$                                 // number of 5-element
                                                         groups
12 for  $j = p$  to  $p + g - 1$                             // sort each group
13   sort  $\langle A[j], A[j + g], A[j + 2g], A[j + 3g], A[j + 4g] \rangle$  in place
14 // All group medians now lie in the middle fifth of  $A[p : r]$ .
15 // Find the pivot  $x$  recursively as the median of the group medians.
16  $x = \text{SELECT}(A, p + 2g, p + 3g - 1, \lceil g/2 \rceil)$ 
17  $q = \text{PARTITION-AROUND}(A, p, r, // partition around the pivot$ 
     $x)$ 
18 // The rest is just like lines 3–9 of RANDOMIZED-SELECT.
19  $k = q - p + 1$ 
20 if  $i == k$ 
21   return  $A[q]$                                 // the pivot value is the
                                                         answer
22 elseif  $i < k$ 
23   return  $\text{SELECT}(A, p, q - 1, i)$ 
24 else return  $\text{SELECT}(A, q + 1, r, i - k)$ 

```

The pseudocode starts by executing the **while** loop in lines 1–10 to reduce the number $r - p + 1$ of elements in the subarray until it is divisible by 5. The **while** loop executes 0 to 4 times, each time rearranging the elements of $A[p : r]$ so that $A[p]$ contains the minimum element. If $i = 1$, which means that we actually want the minimum element, then the procedure simply returns it in line 7. Otherwise, **SELECT** eliminates the minimum from the subarray $A[p : r]$ and iterates

to find the $(i - 1)$ st element in $A[p + 1 : r]$. Lines 9–10 do so by incrementing p and decrementing i . If the **while** loop completes all of its iterations without returning a result, the procedure executes the core of the algorithm in lines 11–24, assured that the number $r - p + 1$ of elements in $A[p : r]$ is evenly divisible by 5.

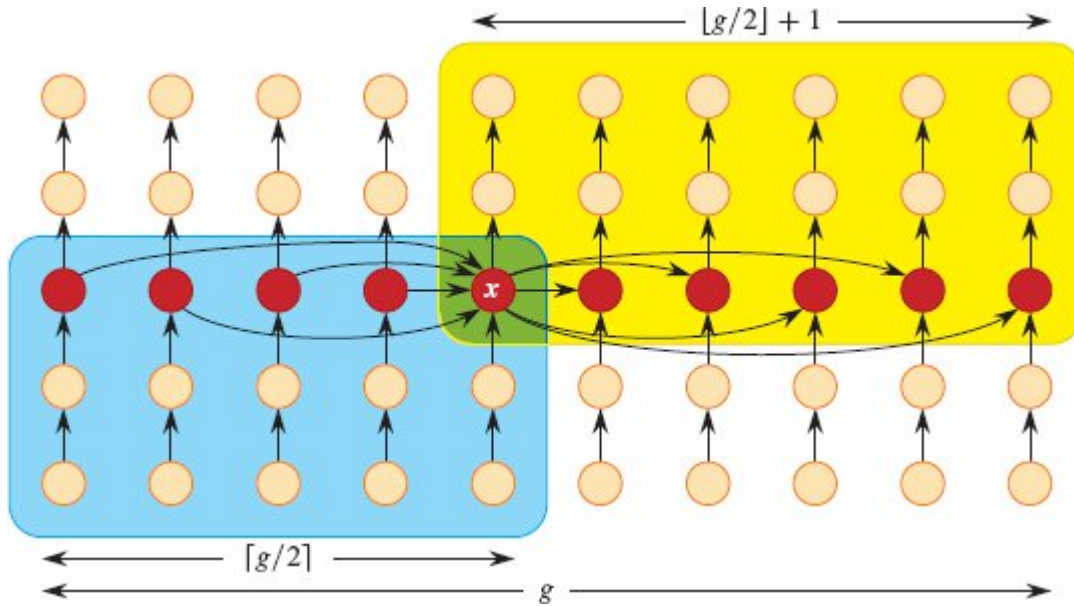


Figure 9.3 The relationships between elements (shown as circles) immediately after line 17 of the selection algorithm SELECT. There are $g = (r - p + 1)/5$ groups of 5 elements, each of which occupies a column. For example, the leftmost column contains elements $A[p]$, $A[p + g]$, $A[p + 2g]$, $A[p + 3g]$, $A[p + 4g]$, and the next column contains $A[p + 1]$, $A[p + g + 1]$, $A[p + 2g + 1]$, $A[p + 3g + 1]$, $A[p + 4g + 1]$. The medians of the groups are red, and the pivot x is labeled. Arrows go from smaller elements to larger. The elements on the blue background are all known to be less than or equal to x and cannot fall into the high side of the partition around x . The elements on the yellow background are known to be greater than or equal to x and cannot fall into the low side of the partition around x . The pivot x belongs to both the blue and yellow regions and is shown on a green background. The elements on the white background could lie on either side of the partition.

The next part of the algorithm implements the following idea, illustrated in Figure 9.3. Divide the elements in $A[p : r]$ into $g = (r - p + 1)/5$ groups of 5 elements each. The first 5-element group is

$\langle A[p], A[p + g], A[p + 2g], A[p + 3g], A[p + 4g] \rangle$,

the second is

$\langle A[p + 1], A[p + g + 1], A[p + 2g + 1], A[p + 3g + 1], A[p + 4g + 1] \rangle$,

and so forth until the last, which is

$\langle A[p + g - 1], A[p + 2g - 1], A[p + 3g - 1], A[p + 4g - 1], A[r] \rangle$.

(Note that $r = p + 5g - 1$.) Line 13 puts each group in order using, for example, insertion sort (Section 2.1), so that for $j = p, p + 1, \dots, p + g - 1$, we have

$A[j] \leq A[j + g] \leq A[j + 2g] \leq A[j + 3g] \leq A[j + 4g]$.

Each vertical column in Figure 9.3 depicts a sorted group of 5 elements. The median of each 5-element group is $A[j + 2g]$, and thus all the 5-element medians, shown in red, lie in the range $A[p + 2g : p + 3g - 1]$.

Next, line 16 determines the pivot x by recursively calling SELECT to find the median (specifically, the $\lceil g/2 \rceil$ th smallest) of the g group medians. Line 17 uses the modified PARTITION-AROUND algorithm to partition the elements of $A[p : r]$ around x , returning the index q of x , so that $A[q] = x$, elements in $A[p : q]$ are all at most x , and elements in $A[q : r]$ are greater than or equal to x .

The remainder of the code mirrors that of RANDOMIZED-SELECT. If the pivot x is the i th largest, the procedure returns it. Otherwise, the procedure recursively calls itself on either $A[p : q - 1]$ or $A[q + 1 : r]$, depending on the value of i .

Let's analyze the running time of SELECT and see how the judicious choice of the pivot x plays into a guarantee on its worst-case running time.

Theorem 9.3

The running time of SELECT on an input of n elements is $\Theta(n)$.

Proof Define $T(n)$ as the worst-case time to run SELECT on any input subarray $A[p : r]$ of size at most n , that is, for which $r - p + 1 \leq n$. By this definition, $T(n)$ is monotonically increasing.

We first determine an upper bound on the time spent outside the recursive calls in lines 16, 23, and 24. The **while** loop in lines 1–10 executes 0 to 4 times, which is $O(1)$ times. Since the dominant time within the loop is the computation of the minimum in lines 2–4, which takes $\Theta(n)$ time, lines 1–10 execute in $O(1) \cdot \Theta(n) = O(n)$ time. The

sorting of the 5-element groups in lines 12–13 takes $\Theta(n)$ time because each 5-element group takes $\Theta(1)$ time to sort (even using an asymptotically inefficient sorting algorithm such as insertion sort), and there are g elements to sort, where $n/5 - 1 < g \leq n/5$. Finally, the time to partition in line 17 is $\Theta(n)$, as Exercise 7.1-3 on page 187 asks you to show. Because the remaining bookkeeping only costs $\Theta(1)$ time, the total amount of time spent outside of the recursive calls is $O(n) + \Theta(n) + \Theta(n) + \Theta(1) = \Theta(n)$.

Now let's determine the running time for the recursive calls. The recursive call to find the pivot in line 16 takes $T(g) \leq T(n/5)$ time, since $g \leq n/5$ and $T(n)$ monotonically increases. Of the two recursive calls in lines 23 and 24, at most one is executed. But we'll see that no matter which of these two recursive calls to SELECT actually executes, the number of elements in the recursive call turns out to be at most $7n/10$, and hence the worst-case cost for lines 23 and 24 is at most $T(7n/10)$. Let's now show that the machinations with group medians and the choice of the pivot x as the median of the group medians guarantees this property.

Figure 9.3 helps to visualize what's going on. There are $g \leq n/5$ groups of 5 elements, with each group shown as a column sorted from bottom to top. The arrows show the ordering of elements within the columns. The columns are ordered from left to right with groups to the left of x 's group having a group median less than x and those to the right of x 's group having a group median greater than x . Although the relative order within each group matters, the relative order among groups to the left of x 's column doesn't really matter, and neither does the relative order among groups to the right of x 's column. The important thing is that the groups to the left have group medians less than x (shown by the horizontal arrows entering x), and that the groups to the right have group medians greater than x (shown by the horizontal arrows leaving x). Thus, the yellow region contains elements that we know are greater than or equal to x , and the blue region contains elements that we know are less than or equal to x .

These two regions each contain at least $3g/2$ elements. The number of group medians in the yellow region is $\lfloor g/2 \rfloor + 1$, and for each group

median, two additional elements are greater than it, making a total of $3(\lfloor g/2 \rfloor + 1) \geq 3g/2$ elements. Similarly, the number of group medians in the blue region is $\lfloor g/2 \rfloor$, and for each group median, two additional elements are less than it, making a total of $3 \lfloor g/2 \rfloor \geq 3g/2$.

The elements in the yellow region cannot fall into the low side of the partition around x , and those in the blue region cannot fall into the high side. The elements in neither region—those lying on a white background—could fall into either side of the partition. But since the low side of the partition excludes the elements in the yellow region, and there are a total of $5g$ elements, we know that the low side of the partition can contain at most $5g - 3g/2 = 7g/2 \leq 7n/10$ elements. Likewise, the high side of the partition excludes the elements in the blue region, and a similar calculation shows that it also contains at most $7n/10$ elements.

All of which leads to the following recurrence for the worst-case running time of SELECT:

$$T(n) \leq T(n/5) + T(7n/10) + \Theta(n) . \quad (9.1)$$

We can show that $T(n) = O(n)$ by substitution.² More specifically, we'll prove that $T(n) \leq cn$ for some suitably large constant $c > 0$ and all $n > 0$. Substituting this inductive hypothesis into the right-hand side of recurrence (9.1) and assuming that $n \geq 5$ yields

$$\begin{aligned} T(n) &\leq c(n/5) + c(7n/10) + \Theta(n) \\ &\leq 9cn/10 + \Theta(n) \\ &= cn - cn/10 + \Theta(n) \\ &\leq cn \end{aligned}$$

if c is chosen large enough that $c/10$ dominates the upper-bound constant hidden by the $\Theta(n)$. In addition to this constraint, we can pick c large enough that $T(n) \leq cn$ for all $n \leq 4$, which is the base case of the recursion within SELECT. The running time of SELECT is therefore $O(n)$ in the worst case, and because line 13 alone takes $\Theta(n)$ time, the total time is $\Theta(n)$.

■

As in a comparison sort (see Section 8.1), SELECT and RANDOMIZED-SELECT determine information about the relative order of elements only by comparing elements. Recall from Chapter 8 that sorting requires $\Omega(n \lg n)$ time in the comparison model, even on average (see Problem 8-1). The linear-time sorting algorithms in Chapter 8 make assumptions about the type of the input. In contrast, the linear-time selection algorithms in this chapter do not require any assumptions about the input's type, only that the elements are distinct and can be pairwise compared according to a linear order. The algorithms in this chapter are not subject to the $\Omega(n \lg n)$ lower bound, because they manage to solve the selection problem without sorting all the elements. Thus, solving the selection problem by sorting and indexing, as presented in the introduction to this chapter, is asymptotically inefficient in the comparison model.

Exercises

9.3-1

In the algorithm SELECT, the input elements are divided into groups of 5. Show that the algorithm works in linear time if the input elements are divided into groups of 7 instead of 5.

9.3-2

Suppose that the preprocessing in lines 1–10 of SELECT is replaced by a base case for $n \geq n_0$, where n_0 is a suitable constant; that g is chosen as $\lfloor (r - p + 1)/5 \rfloor$; and that the elements in $A[5g : n]$ belong to no group. Show that although the recurrence for the running time becomes messier, it still solves to $\Theta(n)$.

9.3-3

Show how to use SELECT as a subroutine to make quicksort run in $O(n \lg n)$ time in the worst case, assuming that all elements are distinct.

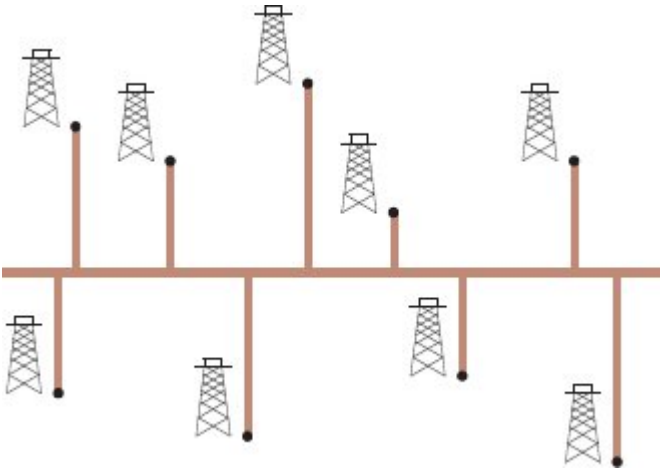


Figure 9.4 Professor Olay needs to determine the position of the east-west oil pipeline that minimizes the total length of the north-south spurs.

★ 9.3-4

Suppose that an algorithm uses only comparisons to find the i th smallest element in a set of n elements. Show that it can also find the $i - 1$ smaller elements and the $n - i$ larger elements without performing any additional comparisons.

9.3-5

Show how to determine the median of a 5-element set using only 6 comparisons.

9.3-6

You have a “black-box” worst-case linear-time median subroutine. Give a simple, linear-time algorithm that solves the selection problem for an arbitrary order statistic.

9.3-7

Professor Olay is consulting for an oil company, which is planning a large pipeline running east to west through an oil field of n wells. The company wants to connect a spur pipeline from each well directly to the main pipeline along a shortest route (either north or south), as shown in Figure 9.4. Given the x - and y -coordinates of the wells, how should the professor pick an optimal location of the main pipeline to minimize the

total length of the spurs? Show how to determine an optimal location in linear time.

9.3-8

The k th *quantiles* of an n -element set are the $k - 1$ order statistics that divide the sorted set into k equal-sized sets (to within 1). Give an $O(n \lg k)$ -time algorithm to list the k th quantiles of a set.

9.3-9

Describe an $O(n)$ -time algorithm that, given a set S of n distinct numbers and a positive integer $k \leq n$, determines the k numbers in S that are closest to the median of S .

9.3-10

Let $X[1 : n]$ and $Y[1 : n]$ be two arrays, each containing n numbers already in sorted order. Give an $O(\lg n)$ -time algorithm to find the median of all $2n$ elements in arrays X and Y . Assume that all $2n$ numbers are distinct.

Problems

9-1 *Largest i numbers in sorted order*

You are given a set of n numbers, and you wish to find the i largest in sorted order using a comparison-based algorithm. Describe the algorithm that implements each of the following methods with the best asymptotic worst-case running time, and analyze the running times of the algorithms in terms of n and i .

- a.* Sort the numbers, and list the i largest.
- b.* Build a max-priority queue from the numbers, and call EXTRACT-MAX i times.
- c.* Use an order-statistic algorithm to find the i th largest number, partition around that number, and sort the i largest numbers.

9-2 *Variant of randomized selection*